

Restoring an image with the Wiener filter

The Wiener filter is **Mean Squared Error (MSE)** filtering that incorporates both the degradation function and the statistical characteristics of noise. The underlying assumption is that the noise and image are uncorrelated. It optimizes the filter so that MSE is minimized. In this recipe, you will learn how to implement the Wiener filter using functions from scikit-image restoration module and how to apply the filter to restore a degraded image, both in a supervised and unsupervised manner.

Getting ready

In this recipe, we shall use a cactus image as input and corrupt it with noise/blur. As usual, let's first import all of the required libraries using the following lines of code:

```
from skimage.io import imread
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import LinearLocator, FormatStrFormatter
```

How to do it...

Follow the steps listed to implement the Wiener filter and apply it to restore images corrupted with blur/noise:

1. Read the image and convert it into grayscale:

```
im = color.rgb2gray(imread('images/cactus.png'))
```

2. Define `convolve2d()` to implement the convolution in the frequency domain (by the convolution theorem, this implementation works much faster than the one in the spatial domain). Construct the 7x7 average (box-blur) point-spread function and use it as convolution-kernel to blur the image, with the following code:

```
def convolve2d(im, psf, k):
    M, N = im.shape
    freq = fp.fft2(im)
    # assumption: min(M,N) > k > 0, k odd
    psf = np.pad(psf, ((M-k)//2, (M-k)//2+1), \
                  ((N-k)//2, (N-k)//2+1)), mode='constant')
    freq_kernel = fp.fft2(fp.ifftshift(psf))
    return np.abs(fp.ifft2(freq*freq_kernel))
```

```
k = 5
psf = np.ones((k, k)) / k**2 # box blur
im1 = convolution2d(im, psf, k)
```

3. Add noise to the blurred image to obtain the degraded image:

```
im1 += 0.2 * im.std() * np.random.standard_normal(im.shape)
```

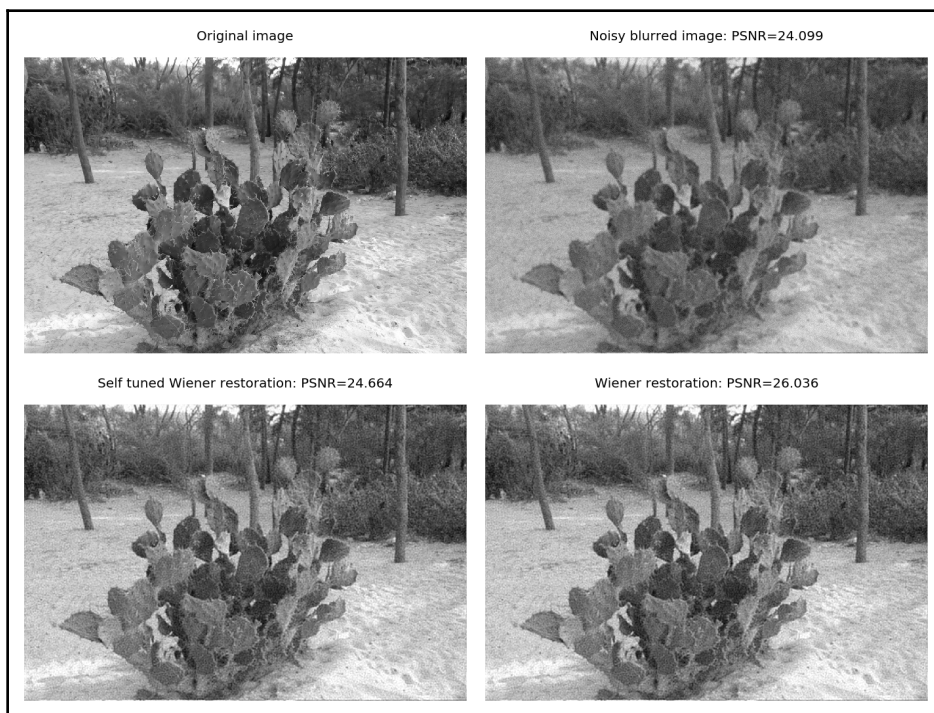
4. Apply the unsupervised Wiener filter on the degraded image along with the kernel as input to restore the image:

```
im2, _ = restoration.unsupervised_wiener(im1, psf)
```

5. Apply the Wiener filter (this time with the `balance` parameter) on the degraded image along with the kernel as input to restore the image:

```
im3 = restoration.wiener(im1, psf, balance=0.25)
```

If you plot the input and output images, you will get a screenshot like this:



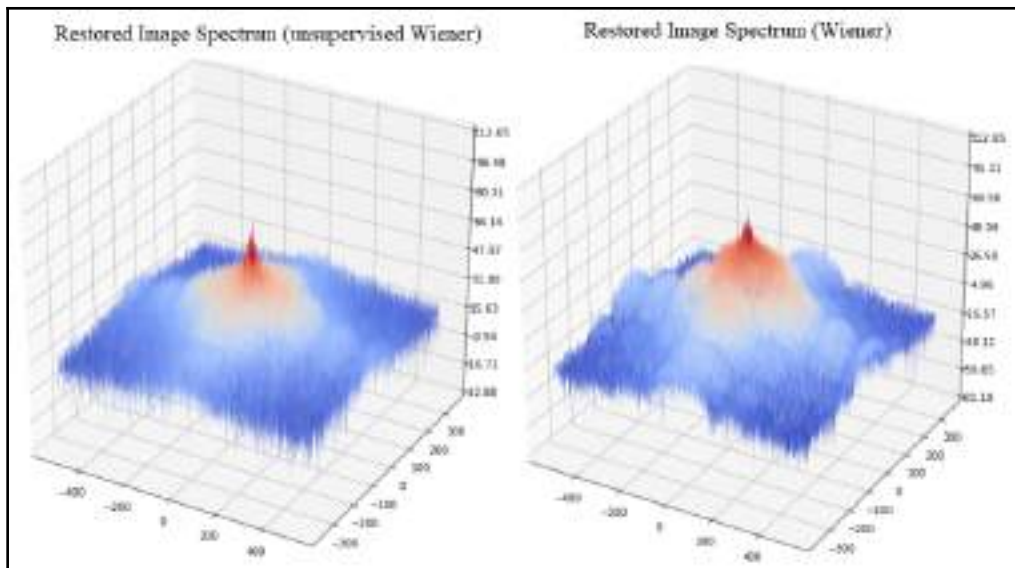
As you can see from the preceding screenshot, the Wiener restoration with the `balance` parameter performs better than the unsupervised Wiener restoration. We measured the restored image quality using the **Peak Signal-to-Noise Ratio (PSNR)** metric.

6. Plot the frequency spectra of the input/output images and kernel in 3D with the following code:

```
def plot_freq_spec_3d(freq):
    fig = plt.figure(figsize=(10,10))
    ax = fig.gca(projection='3d')
    Y = np.arange(-freq.shape[0]//2, freq.shape[0]-freq.shape[0]//2)
    X = np.arange(-freq.shape[1]//2, freq.shape[1]-freq.shape[1]//2)
    X, Y = np.meshgrid(X, Y)
    Z = (20*np.log10( 0.01 + fp.fftshift(freq))).real
    surf = ax.plot_surface(X, Y, Z, cmap=plt.cm.coolwarm, \
                          linewidth=0, antialiased=True)
    ax.zaxis.set_major_locator(LinearLocator(10))
    ax.zaxis.set_major_formatter(FormatStrFormatter('%.02f'))
    plt.show()

plot_freq_spec_3d(fp.fft2(im))
plot_freq_spec_3d(fp.fft2(im1))
plot_freq_spec_3d(fp.fft2(im2))
plot_freq_spec_3d(fp.fft2(im3))
```

The following screenshot is part of the output that you will get after running the preceding code:



How it works...

The following diagram shows how the Wiener filter can be computed. As can be seen, the Wiener filter is the filter that has the objective function of minimizing the MSE between the restored and the original image:

Wiener Filter

objective: $\min_W E[(f - \hat{f})^2]$

$= \min_W E[|F(u, v) - \hat{F}(u, v)|^2]$ by Parseval's Theorem

s.t. $\hat{F}(u, v) = G(u, v)W(u, v)$

solution:

$$W(u, v) = \frac{H^*(u, v)}{|H(u, v)|^2 + \frac{\lambda |u, v|^2}{2^{|\alpha|+|\beta|}}} = \frac{H^*(u, v)}{|H(u, v)|^2 + K}$$

$\frac{1}{|H(u, v)|^2 + \frac{\lambda |u, v|^2}{2^{|\alpha|+|\beta|}}}$ inverse filter $1/\text{SNR}$

$W(u, v) = \frac{H^*(u, v)}{|H(u, v)|^2 + \lambda |A_D|^2}$ action-energy restoration Wiener

balance: Freq. response of Lens/Defocus

As you can see from the preceding diagram, the Wiener filter reduces to an inverse filter when no noise is added to the input image ($N=0$). Here, K is a constant (that is, it does not depend on u, v), to be chosen according to prior knowledge about the noise.

The `unsupervised_wiener()` function from the `scikit-image.restoration` module was used to denoise the degraded image using deconvolution with a Wiener-Hunt approach, where the hyperparameters are automatically estimated. For the unsupervised Wiener algorithm, the estimated image is defined as the posterior mean from a Bayesian analysis. The mean is defined as the total number of all possible images weighted according to their probability. The exact sum is not tractable, hence the **Markov Chain Monte Carlo (MCMC)** method (Gibbs sampler) is used to draw the image under the posterior law.

The Gibbs sampler draws highly probable images more (than the less probable images) since they have a higher contribution to the mean. The empirical mean of these samples is used as an estimation of the mean. The Laplacian is used as a regularization operator by default. The `wiener()` function from the `scikit-image.restoration` module was used, this time with an additional `balance` parameter to denoise the degraded image using the Wiener filter. Again, this filter returns the deconvolution with a Wiener-Hunt approach (that is, with Fourier diagonalization).

The `regularization` parameter (`balance` or λ) value changes the balance between the data and prior adequacy. The data adequacy helps to improve frequency restoration and prior adequacy helps to reduce frequency restoration to avoid noise artifacts. The Laplacian is used as a default regularization operator.

See also

Refer to the following articles to learn more about this recipe:

- https://scikit-image.org/docs/dev/api/skimimage.restoration.html#skimage.restoration.unsupervised_wiener
- <http://sce2.umkc.edu/csee/lizhu/teaching/2018.fall.digital-image-processing/notes/lec11.pdf>